AI-Driven
Construction Documentation System

Course Name: GenAI

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
1.	Vivek Sharma	EN22ME304116
2.	Jayesh Chouhan	EN23MEEL4001
3.	Sakina Modi	EN22EL301044
4.	Vinay Birla	EN22CS3011083
5.	Dev Singh Ghuraiya	EN22ME304022

Group Name:05D10

Project Number:DO-18

Industry Mentor Name:

University Mentor Name: Divya Kumawat

Academic Year:2025-26

Table of Contents

1. Problem Statement & Objectives
   - 1.1. 1.1 Problem Statement
   - 1.2. 1.2 Project Objectives
   - 1.3. 1.3 Scope of the Project
2. Proposed Solution
3. Key Features
4. Overall Architecture / Workflow
   - 4.1. Eraser.ai Flowchart Prompt
5. Tools & Technologies Used
6. Results & Output
   - 6.1. 6.1 Screenshots
   - 6.2. 6.2 Key Outcomes
7. Conclusion
8. Future Scope & Enhancements
9. References

1. Problem Statement & Objectives

1.1 Problem Statement

Construction site managers and engineers spend significant time manually drafting **daily site reports**, **safety inspection logs**, **progress summaries**, and **incident records**. This process is repetitive, error-prone, and inconsistent — varying in tone, structure, and terminology across individuals and teams.

Build a specialized **GenAI content generation tool** tailored for **Construction professionals**. By implementing sophisticated **Prompt Engineering techniques**, the application enables users to transform basic topics into **high-quality professional outputs**, such as '**Site Reports**'. The system ensures **consistency in tone, formatting, and industry-standard terminology**, significantly reducing the manual effort required for drafting complex documentation.

1.2 Project Objectives

- Automate the generation of professional construction site reports using **Generative AI**
- Implement a **Retrieval-Augmented Generation (RAG) pipeline** to ground AI outputs in real construction knowledge, preventing **hallucination**
- Provide **tone and length control** so outputs match formal, standard, or detailed reporting needs
- Enable **multi-turn AI chat** for construction Q&A with full **session memory**
- Deliver a clean, responsive **web interface** requiring no technical knowledge to operate
- Support **multi-format export** (PDF, Word, TXT) for professional distribution

1.3 Scope of the Project

In scope:

- AI-powered **report generation** from structured form inputs
- **RAG pipeline** using a local **ChromaDB vector store** populated with 6 construction knowledge documents
- **Conversational AI assistant** restricted to construction domain topics
- **Client-side report history** with search, view, download, and delete
- **Multi-format export**: PDF (jsPDF), Word (.docx), plain text
- **Responsive single-page web application** (desktop + mobile)

Out of scope:

- **User authentication** and multi-user support
- **Server-side database** or cloud storage
- **Native mobile applications**
- **Real-time collaboration** or report sharing

2. Proposed Solution

ConstructionGen is a **full-stack web application** that combines a **React frontend** with a **Python FastAPI backend** and a **Groq-hosted LLM (LLaMA 3.3 70B)**. The core solution is built around three principles:

1. Structured Input → Grounded Output Rather than a free-text prompt, users fill a structured form (project name, date, location, contractor, weather, tone, length, site activity description). All fields are **injected directly into the prompt**, preventing the model from **fabricating project details**.

2. RAG for Domain Accuracy A **ChromaDB vector store** holds 6 embedded construction knowledge documents (daily report templates, safety guidelines, terminology glossary, QC standards, incident requirements, progress best practices). On every request, the **top 3 semantically similar chunks** are retrieved and **injected as context** — grounding the LLM in real construction knowledge.

3. Tone & Length as Explicit Instructions Each tone/length option maps to a **concrete instruction string** sent to the model:

- **Formal** → third person, no contractions, regulatory-grade language
- **Brief** → under 300 words, 1–2 sentences per section
- **Detailed** → 700+ words, full paragraphs per section

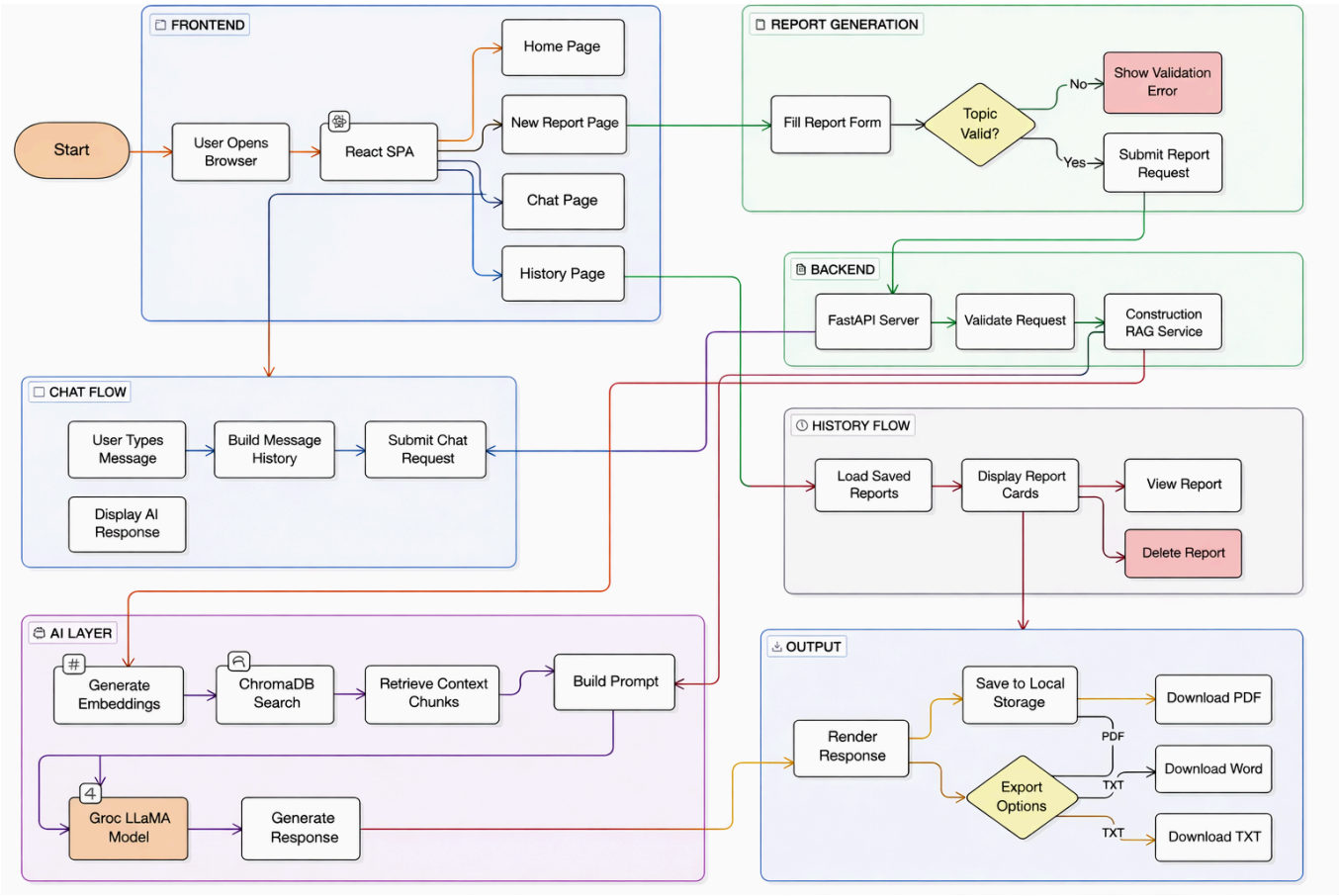
This ensures user selections have a **measurable, consistent effect** on **output quality**.

3. Key Features

- **AI Report Generation** — Generate structured Daily Site Reports, Safety Inspection Reports, Progress Reports, QC Reports, Incident Reports, and more from a single form
- **RAG Pipeline** — ChromaDB vector store with HuggingFace embeddings grounds every response in real construction knowledge
- **Tone & Length Control** — Formal / Standard / Detailed tone; Brief / Standard / Detailed length
- **Smart Section Detection** — Only relevant sections (Safety, Materials, Workforce, QC, Issues) are included based on keyword analysis of the topic description
- **AI Chat Assistant** — Multi-turn conversational assistant with full session memory, restricted to construction domain
- **Report History** — Persistent local storage of all generated reports with search, view, and delete
- **Multi-Format Export** — Download reports as PDF, Word (.docx), or plain text
- **Toast Notifications** — Non-blocking feedback for all user actions
- **Responsive Design** — Works on desktop, tablet, and mobile with a floating sidebar dock

4. Overall Architecture / Workflow

System Architecture



- **Frontend & User Interaction**
  - React SPA with pages: Home, New Report, Chat, and History
  - Handles UI rendering, routing, and user interactions
  - User can:
    - Fill report forms for generation
    - Send chat messages with context
  - Performs basic validation before sending requests

- **Backend & API Layer**
  - Built with FastAPI as the server
  - Receives requests from frontend (report + chat)
  - Validates input and request data
  - Routes requests to the RAG processing service

- **AI / RAG Processing Pipeline**
  - Converts user query into embeddings using HuggingFace models
  - Searches ChromaDB vector store for relevant documents
  - Retrieves context and combines it with user input
  - Constructs a structured prompt
  - Sends prompt to Groq-hosted LLaMA model
  - Generates intelligent, context-aware responses

- **Output, Storage & History**
  - Displays AI-generated response in the frontend
  - Saves reports locally (localStorage)
  - Provides export options:
    - PDF
    - Word
    - TXT
  - Maintains history of reports
  - Allows users to view or delete saved reports

- **Overall Workflow**
  - User provides input via frontend
  - Backend processes and forwards to RAG pipeline
  - Relevant knowledge is retrieved and passed to LLM
  - LLM generates final response
  - Output is displayed, stored, and available for export

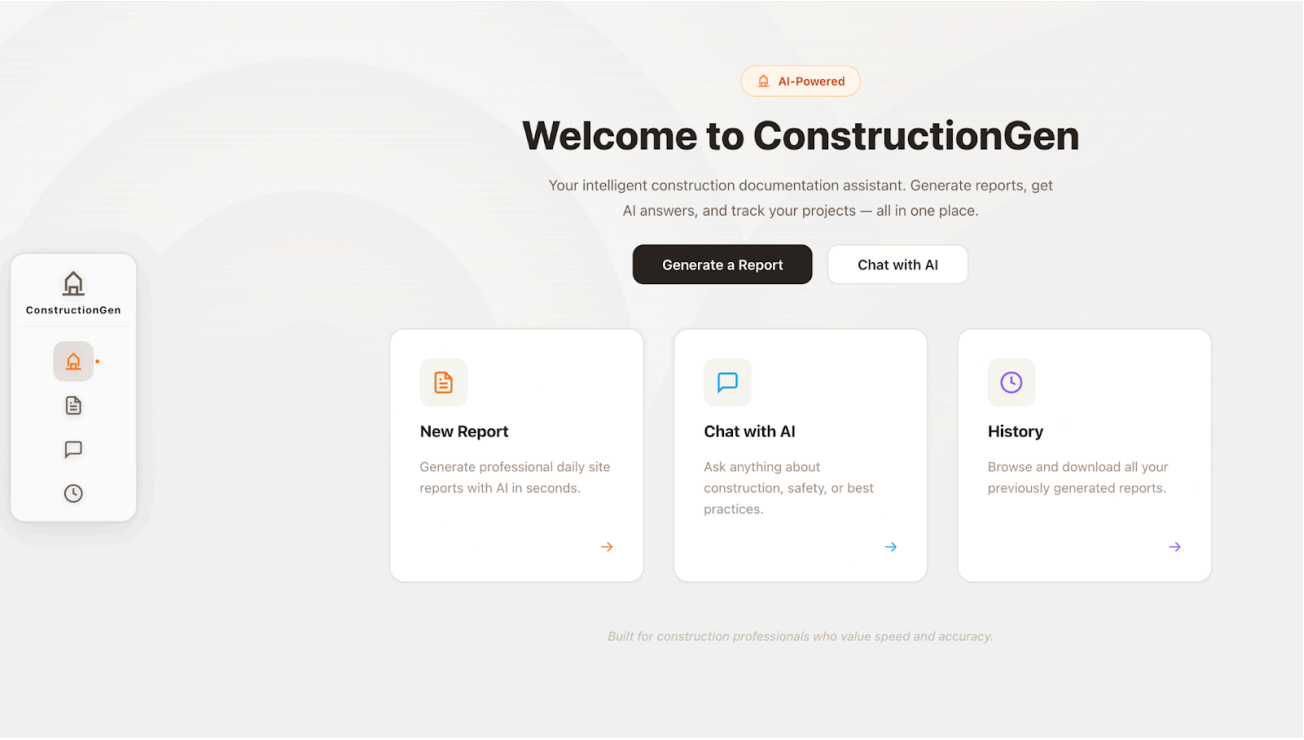
5. Tools & Technologies Used

- **Frontend**: React 18 + TypeScript (SPA) with Vite 5
- **Routing**: React Router v6 for navigation
- **Markdown**: react-markdown + remark-gfm for AI reports
- **Exports**: jsPDF (PDF) + docx & file-saver (Word)
- **Backend**: FastAPI + Pydantic with Uvicorn server
- **LLM Stack**: Groq API (LLaMA 3.3 70B) + LangChain
- **Vector DB**: ChromaDB with HuggingFace embeddings
- **Config**: python-dotenv for environment variables
- **Styling**: Custom CSS (no Tailwind)
- **Storage**: localStorage for reports & chat history

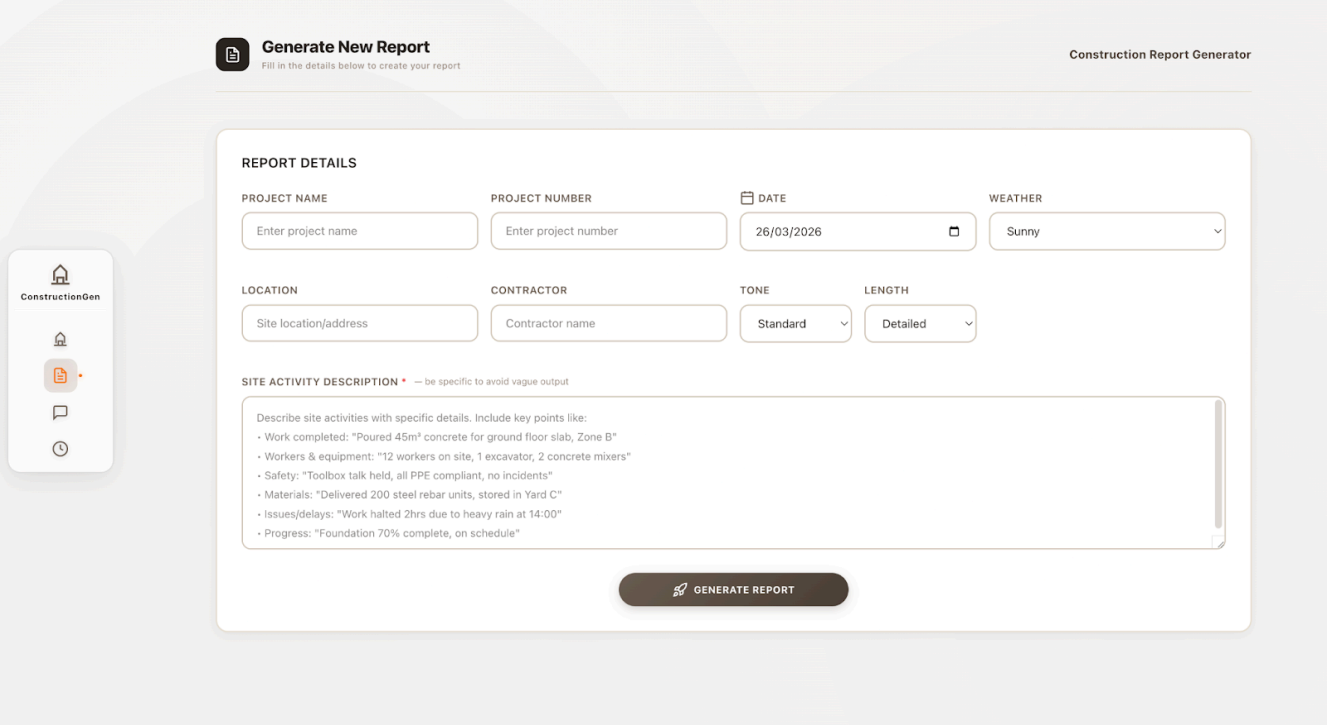
6. Results & Output

6.1 Screenshots

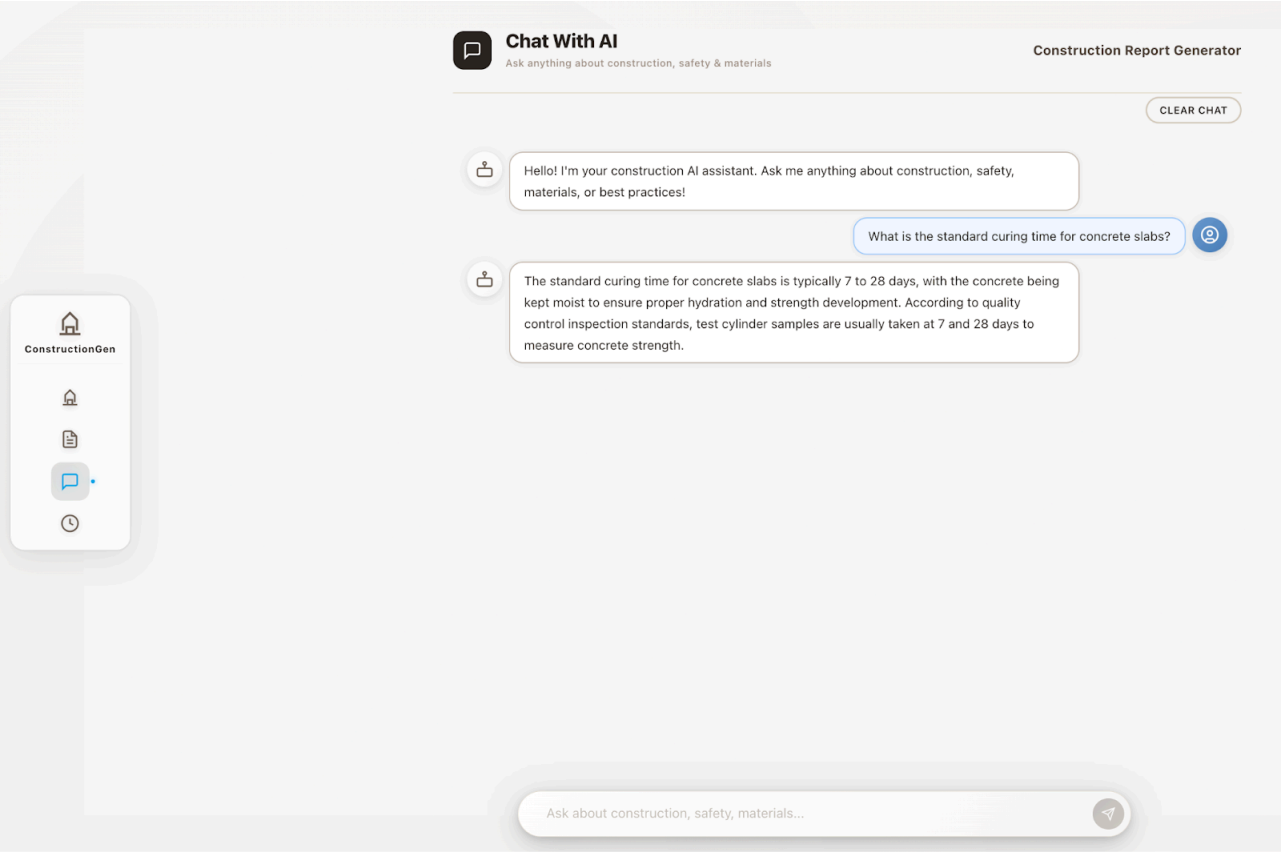
- **Home Page**



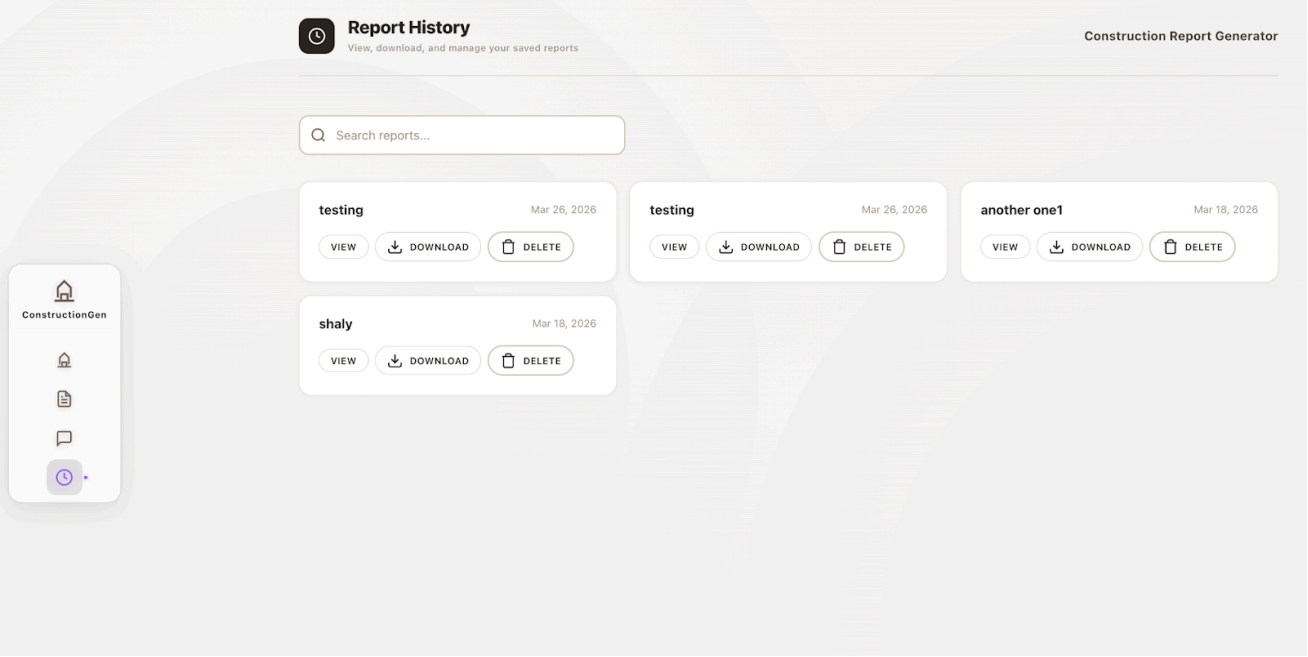
- **New Report Page — Form**



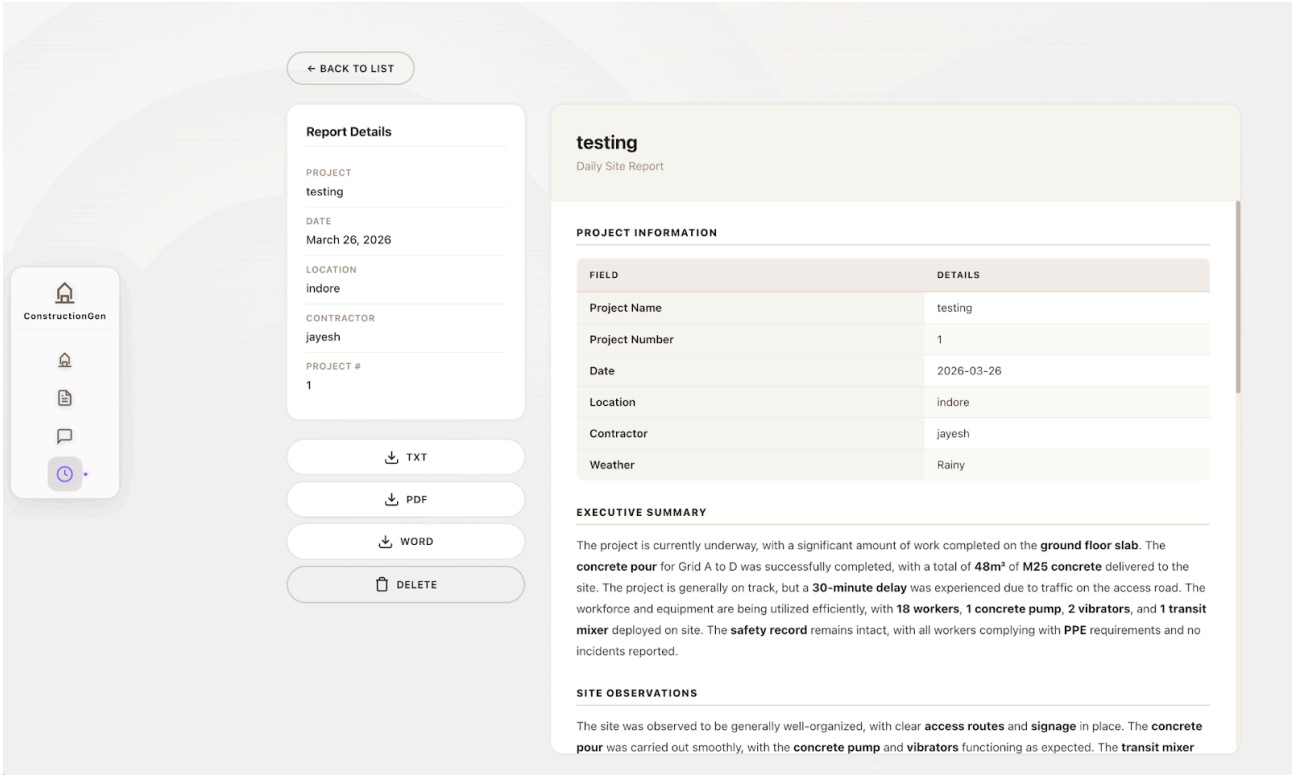
- **Chat With Ai Page**



- **History Page — Report List**



- **History Page — Report Detail Viewer**



6.2 Key Outcomes

- **Report generation** completes in under 5 seconds on average using **Groq's LPU inference hardware**
- **RAG pipeline** successfully retrieves relevant **construction knowledge chunks** for every request, grounding outputs in domain-specific content
- **Smart section detection** ensures only contextually relevant sections appear in each report — no generic filler sections
- **Tone and length controls** produce measurably different outputs: brief reports stay under 300 words, detailed reports exceed 700 words consistently
- **Multi-turn chat** correctly maintains conversation context across messages within a session
- All three **export formats (PDF, Word, TXT)** produce clean, professional documents with markdown symbols stripped
- **Zero backend storage** required — full CRUD on reports via **localStorage** with no database dependency

7. Conclusion

ConstructionGen demonstrates how **Generative AI**, when combined with **structured prompt engineering** and a domain-specific **RAG pipeline**, can meaningfully reduce the manual effort involved in **professional documentation**. By collecting **structured project metadata** through a **form** and injecting it directly into a carefully engineered **prompt** — alongside retrieved **construction knowledge** from **ChromaDB** — the system produces consistent, accurate, and professionally formatted reports without fabricating details.

The key learning from this project is that the quality of GenAI output is determined far more by **prompt design** and **input structure** than by model size alone. Explicit **tone/length instructions**, strict "do not fabricate" rules, and RAG-grounded context together produce outputs that are reliable enough for real professional use. The **stateless backend architecture** and **client-side persistence model** also demonstrate that a production-quality AI application can be built without a **database**, keeping the system simple, fast, and easy to deploy.

8. Future Scope & Enhancements

- **User Authentication**: JWT-based login for multi-user report management
- **Database**: Use PostgreSQL/MongoDB for persistent storage (instead of localStorage)
- **Streaming Responses**: Real-time report generation using Groq streaming API
- **Image Uploads**: Attach site photos in reports (PDF/Word export)
- **Custom Knowledge Base**: Upload project docs to enhance RAG system
- **Batch Reports**: Auto-generate weekly/monthly summaries
- **Email Integration**: Send reports directly to stakeholders
- **Offline Support**: PWA with caching (except LLM calls)
- **Multi-language**: Generate reports in different languages
- **Analytics Dashboard**: Track usage, reports, and activity
- **CI/CD & Deployment**: Docker + deploy (Railway/Render + Vercel)

9. References

- Meta AI. (2024). ***LLaMA 3: Open Foundation and Fine-Tuned Chat Models***. Meta AI Research. https://ai.meta.com/llama/
- Groq Inc. (2024). ***Groq Cloud API Documentation — LPU Inference Engine***. https://console.groq.com/docs/
- Lewis, P., Perez, E., Piktus, A., et al. (2020). ***Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks***. NeurIPS 2020. https://arxiv.org/abs/2005.11401
- LangChain. (2024). ***LangChain Python Documentation***. https://python.langchain.com/docs/
- Chroma. (2024). ***ChromaDB — The AI-native open-source embedding database***. https://docs.trychroma.com/
- Reimers, N., & Gurevych, I. (2019). ***Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks***. EMNLP 2019. https://arxiv.org/abs/1908.10084
- HuggingFace. (2024). ***sentence-transformers/all-MiniLM-L6-v2 Model Card***. https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2
- Ramírez, S. (2024). ***FastAPI Documentation***. https://fastapi.tiangolo.com/
- Meta Open Source. (2024). ***React 18 Documentation***. https://react.dev/
- ViteJS. (2024). ***Vite — Next Generation Frontend Tooling***. https://vitejs.dev/
- Remix Software. (2024). ***React Router v6 Documentation***. https://reactrouter.com/
- Silentmatt. (2024). ***react-markdown — Markdown component for React***. https://github.com/remarkjs/react-markdown
- Remarkjs. (2024). ***remark-gfm — GitHub Flavored Markdown plugin***. https://github.com/remarkjs/remark-gfm
- dolanmiu. (2024). ***docx — Generate .docx files with JavaScript***. https://docx.js.org/
- MrRio. (2024). ***jsPDF — Client-side JavaScript PDF generation***. https://github.com/parallax/jsPDF
- Occupational Safety and Health Administration (OSHA). (2024). ***Construction Industry Standards (29 CFR Part 1926)***. U.S. Department of Labor.https://www.osha.gov/construction
- American Concrete Institute. (2019). ***ACI 301-16: Specifications for Structural Concrete***. ACI International.
- Construction Industry Institute. (2023). ***Best Practices for Daily Site Reporting and Documentation***. University of Texas at Austin.